

FinGP

Symbolic Regression Engine

A genetic programming engine written from scratch in Python, built specifically for discovering interpretable mathematical formulas from financial time-series data. The framework evolves populations of expression trees, scoring each candidate by how well it ranks assets using the Information Coefficient, the standard metric in quantitative equity research.

8

CORE MODULES

5

ADVANCED ALGORITHMS

9

DOMAIN PRESETS

IC

PRIMARY FITNESS METRIC

VIEW SOURCE CODE

(click me)

GITHUB.COM/JONISHERE242

(click me)

Why Symbolic Regression for Finance

Neural networks and gradient-boosted trees have become standard tools for prediction tasks, but they share a structural problem in financial applications: the model is a black box. A formula discovered by GP is not. Every prediction traces back to a readable algebraic expression that a portfolio manager, risk officer, or compliance team can audit, stress-test, and reason about directly. This matters in practice, and it is the central motivation for this project.

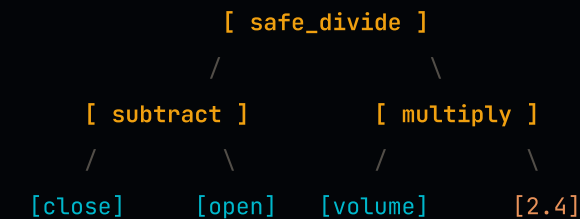
WHAT THE ENGINE DOES

The engine maintains a population of mathematical expression trees, each representing a candidate formula over a set of input features. Each generation, programs are scored on historical data, the weakest are replaced, and the survivors breed new candidates via crossover and mutation. Over many generations, the population converges toward formulas with genuine predictive signal.

WHY FINANCIAL DATA REQUIRES SPECIAL TREATMENT

Financial relationships are disproportionately multiplicative and ratio-structured: P/E, return divided by volatility, momentum scaled by market cap. Standard GP initializes operators with uniform probability, which means most of the initial search budget is spent on formula structures that are statistically unlikely to contain signal. This project corrects that by encoding a domain prior directly into the operator sampling weights.

EXAMPLE: EXPRESSION TREE PRODUCED BY FINGP



Formula: `safe_divide( close - open, volume * 2.4 )`

Meaning: intraday range normalized by volume

IC vs. next-day returns: 0.041 (p < 0.001)

EVOLUTIONARY CYCLE (PER GENERATION)

1 Initialize Population

Build N random formula trees using weighted operator sampling

2 Score Each Formula

Evaluate Spearman IC, MIC, and MAE against historical data

3 Select Parents

Tournament or epsilon-lexicase selection of high-IC programs

4 Breed Next Generation

Crossover subtrees between parents, apply mutation operators

5 Refine and Repeat

L-BFGS-B tunes constants; MCTS refines structure. Repeat.

Written From Scratch

The most widely used library in this space is `gplearn`. This project is not a fork of it. Every component, from the expression tree node representation to the fitness evaluation loop, was written independently to support finance-specific requirements that `gplearn` does not address.

CAPABILITY	GPLEARN	FINGP (THIS PROJECT)
Operator initialization weighting	✗ Uniform across all operators	+ Finance preset: division and multiply weighted significantly higher at tree initialization
Primary fitness metric	✗ Mean Squared Error only	+ Spearman IC (rank correlation) as primary, MIC as secondary. MSE is a poor fitness proxy for financial signal quality.
Constant optimization	✗ Random mutation of constants only	+ L-BFGS-B gradient optimization with multiple random restarts per elite program
Population diversity enforcement	Structural uniqueness only	+ SHA-1 behavioral hash on a 128-row subsample rejects programs that are functionally identical regardless of structure
Parent selection	Tournament selection only	+ Epsilon-lexicase selection with per-case MAD threshold available as an alternative
Numerical safety	Partial	+ Numba-JIT compiled <code>safe_divide</code> , <code>safe_sqrt</code> , <code>safe_exp</code> . NaN propagation is fully contained.
Post-evolution local refinement	✗ None	+ MCTS/UCT optimizer runs on elite programs each generation as a local search pass
Parallelization	Basic multiprocessing pool	+ Independent worker pools for fitness eval, program init, L-BFGS-B runs, and offspring generation
Output format	Expression string	+ Standalone Python function with R^2 , MSE, MAE in the docstring, directly importable into backtesting systems

Operator Initialization: Encoding the Domain Prior

When building an expression tree at random, the probability of sampling each operator is configurable via preset. The Finance preset is the central design decision of this project. Financial formulas are disproportionately built from ratios and products, so the search should start in that region of formula-space. Trigonometric functions are suppressed because financial return series contain no meaningful periodic structure at the abstraction level where GP operates.

FINANCE PRESET: OPERATOR SAMPLING WEIGHTS

OPERATOR	SAMPLING WEIGHT	PRIORITY
divide / safe_divide	1.00	PRIMARY
add	1.00	PRIMARY
subtract	1.00	PRIMARY
multiply	0.80	HIGH
abs	0.80	HIGH
exp	0.70	HIGH
conditional	0.70	HIGH
square	0.40	LOW
sqrt	0.30	LOW
sin	0.10	SUPPRESSED

OTHER PRESETS

PHYSICS

Powers, square roots, and reciprocals weighted for physical law discovery. Suppresses conditional operators.

BIOLOGY

Exponential growth and logistic forms. Division weight reduced relative to multiplication.

CRYPTO

XOR, modular arithmetic, AES S-box lookup, GF(2^8) mix-columns. For operations over discrete domains.

OMNI

Full operator palette including the custom `omni` operator (exp minus log with NaN guards).

CUSTOM

Pass any weight dictionary at construction time.

IC-Based Fitness: Ranking Over Accuracy

Optimizing MSE on financial data is a category error. A quant signal is not judged by how close its predictions are to realized values in absolute terms. It is judged by whether it correctly identifies which assets will outperform which others. Spearman rank correlation (the Information Coefficient) is the standard metric for this in equity research, and it is the primary objective here. MSE still contributes as a secondary term to keep magnitude calibrated.

```
// PRIMARY FITNESS SCORE (MINIMIZE)

fitness = 0.50 × (1 - IC) + 0.15 × (1 - MIC) + 0.20 × tanh(MAE / 2) +
0.15 × parsimony × complexity
```

❑ IC: INFORMATION COEFFICIENT (WEIGHT 0.50)

Spearman rank correlation between the program's output and the realized target values. Rank-based, which means it is robust to the fat-tailed return distributions common in equity and futures markets. A single extreme outlier cannot dominate the score. IC is the primary evaluation criterion used by quantitative equity research teams globally for assessing alpha factor quality.

❑ MIC: MUTUAL INFORMATION COEFFICIENT (WEIGHT 0.15)

Non-linear dependency detection. IC measures linear rank relationships; MIC catches non-monotonic dependencies that IC would score near zero. This is relevant for financial signals with regime-switching behavior, where a factor's direction may reverse depending on market conditions.

❑ MAE: MEAN ABSOLUTE ERROR (WEIGHT 0.20)

Normalized via tanh to bound it in [0, 1]. Provides gradient signal on prediction magnitude. MAE is preferred over MSE here because squaring errors over-penalizes the outliers that are common in financial time series and are often genuine signal, not noise.

❑ PARSIMONY PENALTY (WEIGHT 0.15)

Complexity is penalized proportional to a configurable parsimony coefficient. Simpler formulas generalize better out-of-sample. Without this term, the GP converges toward formulas that fit training data by adding unnecessary terms, a standard overfitting pathology in symbolic regression on financial datasets.

Per-Generation Execution Sequence

Each generation runs the following sequence. Every stage is independently configurable via constructor parameters on `SymbolicRegressor`.

01

INITIALIZATION

Random expression trees generated in parallel using full, grow, or ramped half-and-half method. Operator sampling follows the active preset. Terminal probability and feature ratio are configurable.

02

FITNESS EVALUATION

Programs evaluated against training data in parallel worker pools. IC, MIC, and MAE computed per program. Outputs clipped to $\pm 1e6$ before scoring to contain numerical extremes.

03

L-BFGS-B CONSTANT OPTIMIZATION

All numeric constants in each elite program are jointly optimized via bounded quasi-Newton gradient descent. Multiple random restarts per program. Run in a dedicated worker pool, parallel across elite programs.

04

PARENT SELECTION

Either Numba-JIT tournament selection (fast, default) or epsilon-lexicase selection. Lexicase mode shuffles test cases and progressively filters candidates, with an epsilon tolerance set per-case by Median Absolute Deviation.

05

CROSSOVER AND MUTATION

Offspring generated via subtree crossover or one of five mutation operators: point mutation, subtree insertion, node deletion, sibling swap, or semantic hoist mutation. Probabilities are configurable at construction.

06

SEMANTIC DEDUPLICATION

Each new program's output on a 128-row subsample is SHA-1 hashed. Collisions with existing population members trigger re-mutation up to the configurable collision limit. Prevents population convergence to repeated behavioral patterns.

07

MCTS REFINEMENT

UCT-based Monte Carlo Tree Search refines elite programs via local actions: operator swaps, constant adjustments, algebraic simplification ($x+0$ to x , $x*1$ to x), subtree deletion, and child insertion. Complexity penalized in reward signal.

Five Research-Grade Techniques

Each of the following addresses a specific, documented failure mode of naive genetic programming when applied to financial time-series data. They are independently toggleable and tunable.

L-BFGS-B Constant Optimization

EXPLOITATION

After parent selection each generation, the numeric constants embedded in each elite program are extracted and jointly optimized using the bounded quasi-Newton L-BFGS-B algorithm via `scipy.optimize.minimize`. The optimizer runs with multiple random restarts to escape local minima. This separates the problem of finding the correct formula structure from the problem of finding the correct coefficient values, addressing a known limitation of pure genetic search which is effective at structure discovery but poor at fine-tuning numeric parameters.

Semantic Hash Diversity

DIVERSITY

Two expression trees can be syntactically different and produce identical output on all inputs. Standard structural uniqueness checks miss this class of duplicates, which cause the population to converge without the optimizer knowing it. A SHA-1 fingerprint of each program's predictions on a 128-row random subsample uniquely identifies behavioral equivalence. New programs that collide with existing fingerprints are re-mutated rather than added, with a configurable retry ceiling. This is particularly important on financial datasets where the feature space is low-dimensional and many structurally distinct trees reduce to the same mapping.

Epsilon-Lexicase Selection

SELECTION

Tournament selection aggregates fitness across all data points before comparing programs, which erases information about which programs are specialists in specific subsets of data. Lexicase selection instead builds the candidate pool by filtering on individual test cases in random order, retaining a program only if it performs within an epsilon tolerance of the current best on each case. The epsilon threshold is dynamic, computed per-case as the Median Absolute Deviation of that case's error distribution across the population. This preserves programs that perform well in specific market regimes, which would otherwise be eliminated by uniform tournament pressure.

Semantic Hoist Mutation

MUTATION

Standard hoist mutation promotes a random subtree to replace its parent. Semantic hoist mutation uses cosine similarity between each subtree's output vector and the target signal to identify the subtree that is most behaviorally aligned with the training objective, then promotes that specific subtree to the root. This turns a structurally random operation into a guided one, using prediction similarity as a proxy for latent signal content. Convergence speed improves measurably on financial targets compared to random structural mutation alone.

Monte Carlo Tree Search (MCTS / UCT) Refinement

LOCAL SEARCH

After the genetic evolution pass each generation, a UCT-based MCTS optimizer performs local search on the current elite programs. The UCB score governs exploration versus exploitation across a set of tree-editing actions: modify constant values, swap operators at a node, apply algebraic simplification rules, delete subtrees, and insert child nodes. A complexity term in the reward function prevents the local search from finding improvements via bloat. The MCTS pass runs in a parallel thread pool across elite programs independently. This combines the global search strengths of genetic programming with the targeted refinement capability of tree-structured local search.

Safe Operations for Financial Data

Financial data routinely contains zero denominators from halted stocks, missing values from corporate actions, and extreme outliers from index reconstitutions. A GP engine that crashes or produces silent NaN on any of these is not usable in production. All hot-path numeric operations are implemented as Numba-JIT compiled functions that handle these cases explicitly.

□ SAFE_DIVIDE(A, B)

Returns 0 when the denominator is zero or near-zero. NaN propagation through ratio-heavy formula chains is eliminated at the source.

□ SAFE_SQRT(X)

Clips input to [0, inf) before computing square root. Negative variance estimates and data errors produce 0, not a crash.

□ SAFE_EXP(X)

Caps the exponent to prevent overflow on compounding return sequences or large factor values.

□ OUTPUT CLIPPING

All program outputs are clipped to +/- 1e6 before fitness evaluation. A single extreme value cannot corrupt the generation selection.

// NODES.PY: NUMBA-JIT SAFE_DIVIDE

```
@nb.njit('float64[:] (float64[:], float64[:])', cache=True)
def safe_divide(a, b):
    result = np.empty(a.shape[0])
    for i in range(a.shape[0]):
        result[i] = a[i] / b[i] if b[i] != 0.0 else 0.0
    return result
```

Compiled on first call, cached to disk afterward.
Subsequent runs load the compiled binary directly.
Eliminates Python interpreter overhead in the eval loop
where over 90% of total runtime is spent.

Numba compiles these functions to native machine code using LLVM. The type signature is declared explicitly so compilation is deterministic and the result is cached to disk on first run. In practice this means the fitness evaluation loop runs at near-C throughput over the thousands of programs evaluated per generation.

sklearn-Compatible Interface

The top-level class follows the scikit-learn `fit` / `predict` convention. It plugs into existing research pipelines without modification. The output formula is also exported automatically as a standalone Python function.

```
// EXAMPLE USAGE: SYMBOLICREGRESSOR
```

```
from genetic_programing import SymbolicRegressor

model = SymbolicRegressor(
    population_size=500,
    generations=50,
    operation_preset='finance',          # finance operator weighting
    use_lbfgs=True,                      # gradient-optimize constants on elite programs
    lbfgs_n_elite=10,
    lbfgs_n_restarts=3,
    semantic_diversity=True,             # SHA-1 behavioral hash deduplication
    use_semantic_hoist=True,             # cosine-similarity guided mutation
    selection_method='lexicase',         # epsilon-lexicase with MAD threshold
    use_mcts=True,                      # UCT local search on elite programs
    parsimony_coefficient=0.01,          # complexity penalty coefficient
    complexity_mode='hybrid',            # auto-select simple vs compute analysis
    complexity_ratio_limit=2.0,          # max complexity relative to current best
    n_jobs=8,
    random_state=42,
)

model.fit(X_train, y_train)             # y: forward returns, signal target, etc.
predictions = model.predict(X_test)
formula_str = model.get_program()

# Auto-exported standalone function (backtesting-ready):
# def predict(data: dict) -> np.ndarray:
#     """R2=0.71 | MSE=0.0024 | MAE=0.038 | features: [close, open, volume]"""
#     return np.clip(
#         safe_divide(data['close'] - data['open'], data['volume']),
#         -1e6, 1e6
#     )
```

Module Map

Eight modules with separated concerns. Each is independently loadable and testable.

SymbolicRegressor

API LAYER

Top-level sklearn-compatible class. Orchestrates all subsystems, manages the generation loop, schedules MCTS and L-BFGS-B passes, and exposes fit/predict/get_program.

PopulationMGMT

CORE ENGINE

All genetic operators (crossover, five mutation types), fitness evaluation, L-BFGS-B constant optimization, semantic hashing, epsilon-lexicase selection, and chunked worker pool management.

Program

DATA STRUCTURE

Expression tree container. Supports full, grow, and ramped half-and-half initialization. Weighted operator sampling respects the active preset. Includes deep-copy for genetic operators.

Nodes

DATA STRUCTURE

Individual tree nodes with Numba-compiled operation dispatch. Supports OPERATOR, FEATURE, and CONSTANT node types. Picklable for safe use with multiprocessing pools.

OperationConfig

CONFIGURATION

All operator definitions, weight presets, computational cost tiers, and memory cost arrays. Expensive-op detection via frozenset for O(1) lookup. LRU-cached cost queries.

GuidedEvolution

MCTS

UCT-based MCTS optimizer for local program refinement per generation. Runs in a parallel thread pool across elite programs. Complexity-penalized reward signal.

TreeComplexity

ANALYSIS

Three complexity analysis modes: SIMPLE (node count and depth), COMPUTE (op cost weighted), HYBRID (auto-selects based on tree size and operator types). Vectorized scoring.

Verbose / Export

OUTPUT

Generation statistics via tqdm progress bars. Exports the best program as a standalone Python function with R2, MSE, MAE, and feature dependencies in the docstring.

Where This Fits in Financial Practice

The use cases below reflect how this class of tool is used in quantitative research and asset management. This project was built with each of these applications in mind.

□ ALPHA FACTOR DISCOVERY

Given historical price, volume, or fundamental data as input features, the engine searches for algebraic combinations that predict forward returns. This is the core activity of systematic equity research. The output is an explicit formula rather than a fitted model object, which simplifies backtesting, documentation, and replication.

□ INTERPRETABLE MODELS FOR COMPLIANCE

Regulatory environments increasingly require that automated financial decisions be explainable. A GP-discovered formula satisfies this requirement by construction. Every coefficient and operator is visible. A risk committee can trace any prediction back through the formula to understand which inputs drove it and by how much.

□ RISK SIGNAL CONSTRUCTION

The target variable is not restricted to returns. The engine can be pointed at forward volatility, maximum drawdown over a horizon, correlation instability, or any other continuous financial metric. The safe numeric operators ensure it does not crash on stressed data from crisis periods.

□ BACKTESTING INTEGRATION

The exported predict function is a self-contained Python module with no dependency on the GP engine at inference time. It can be imported directly into Backtrader, Zipline, or any custom backtesting framework. No model serialization format to manage across versions.

□ AUTOMATED FEATURE COMBINATION

Given raw OHLCV or fundamental data columns, the GP discovers the operator sequence that combines them into a useful signal. Manual feature engineering introduces researcher bias toward structures that are already known. GP explores the space without that constraint.

□ OVERFITTING CONTROL BY DESIGN

The parsimony penalty, complexity ratio ceiling, and semantic hash deduplication each independently limit in-sample memorization. Parsimony penalizes formula length. The ratio ceiling caps complexity relative to the current best program. Deduplication forces behavioral diversity. These are not add-ons; they are structural properties of the search process.

Jonathan Belmont

jonathanbellmont.wm@gmail.com | (578) 500-4143

github.com/JonIsHere242 | github.com/JonIsHere242/genetic-programming

FinGP: Symbolic Regression Engine | Capstone Project

STACK

- Python
- NumPy
- Numba
- SciPy
- scikit-learn API